SIMATS ENGINEERING

Engineer to Excel

SIMATS

Saveetha Institute of Medical And Technical Sciences

(Declared as Deemed to be University under Section 3 of UGC Act 1956)

MOBILE OS SECURITY & PERMISSIONS AUDIT

PROJECT TITLE

"Secure Mobile Application Framework"

PRESENTED BY

B.Hari Sai-192311098

S.Dinesh-192311302

INDEX

1 Abstract

2 Introduction

3 Problem and Objectives

4 Existing and Proposed System

5 Methodology

6 Technologies Used

7 Modules

8 Results

9 IEEE Standards

10 SDG Contribution

11 Conclusion

12 References

ABSTRACT

- A banking-style Android app is built with secure coding practices and audited for OS security and permission weaknesses.
- The audit covers permissions, data storage, encryption, authentication and network security.
- MobSF and manual tests find vulnerabilities, which are fixed and re-scanned.
- A before-and-after security score documents the improvement.



Data storage



Encryption



Authentication



Network security

INTRODUCTION

- Mobile banking apps are prime targets for attackers.
- Weak storage, permissions, login and network traffic cause most breaches.
- This project builds a sample app and audits it with MobSF.
- Every finding is fixed and re-tested.

Why it matters



Protects user credentials



Keeps data safe at rest and in transit



Finds flaws before attackers do



Aligns with OWASP Mobile Top 10

PROBLEM STATEMENT AND OBJECTIVES

Problem Statement

- Apps request excessive permissions.
- Sensitive data is stored in plain text.
- Weak login and open traffic expose users.
- Security is hard to measure and prove.

Objectives

Secure storage and login:

- Encrypt data; enforce PIN/biometric.

Hardening:

- Apply SSL pinning; limit permissions.

Audit:

- Run MobSF, fix, compare scores.

EXISTING SYSTEM AND PROPOSED SYSTEM

Existing System

- Data stored unencrypted.
- Simple or no authentication.
- Weak HTTPS validation.
- Excessive permissions.
- No measurable security score.

Proposed System

- Encrypted storage with Keystore keys.
- PIN plus biometric login.
- SSL pinning and strict network config.
- Least-privilege permissions.
- MobSF audit with score comparison.

METHODOLOGY

1. Build banking app
2. Baseline MobSF scan
3. Manual security tests
4. Fix vulnerabilities
5. Re-scan and re-test
6. Compare scores

Task-Specific Parameters

- Permission set
- Data storage type
- Encryption algorithm
- Authentication method
- Network security config

Outcome Parameters

- Vulnerability count
- Security score
- Encryption overhead
- Auth success/failure rate

TECHNIQUES AND TOOLS USED

Android Keystore

Hardware-backed key storage

Biometric Authentication

Fingerprint or face with PIN fallback

SSL Pinning

Trusts only the expected server certificate

EncryptedSharedPreferences

Encrypted keys and values

Static Analysis (MobSF)

Automated scan of code and manifest

OWASP Mobile Top 10

Reference list of top mobile risks

MODULE 1: Secure Storage and Encryption

- Protects data saved on the device from theft or malware.
- Uses EncryptedSharedPreferences instead of plain-text preferences.
- Keys are generated and kept in the Android Keystore.
- Uses AES-256 and records the overhead it adds.



Key Functions

- Encrypts local data
- Manages keys
- Measures overhead

MODULE 2: Authentication (PIN / Biometric)

- Only the legitimate user can open the app or approve transactions.
- PIN plus fingerprint or face unlock via the biometric API.
- Limits repeated attempts to block brute force.
- Logs results to compute success and failure rate.



Key Functions

- PIN login
- Biometric prompt
- Attempt limiting
- Success/failure logs

RESULTS

Vulnerability count

Expected to fall after fixes

Security score

Expected to rise in re-scan

Encryption overhead

Measured to confirm low impact

Authentication rate

Success and failure tracked

- MobSF scans the app before and after the fixes; results are documented side by side.
- Manual tests confirm encrypted storage, resilient login and rejected intercepted traffic.
- The before-and-after score shows the gain from secure coding.

IEEE STANDARDS

IEEE 1363	Public-key cryptography for secure key exchange, encryption and signatures.
IEEE 802.1X	Port-based network access control and authentication.
IEEE 2410	Biometric Open Protocol for secure biometric device communication.
IEEE 1619	Cryptographic protection of data on storage devices.

SDG CONTRIBUTION

9

SDG 9

Industry, Innovation and Infrastructure

Promotes secure digital infrastructure for mobile banking.

16

SDG 16

Peace, Justice and Strong Institutions

Protects personal data and builds trust in digital services.

8

SDG 8

Decent Work and Economic Growth

Builds confidence in digital financial services.

CONCLUSION

- Secure coding turns a typical banking-style app into a hardened one.
- Encrypted storage, strong login and SSL pinning close common attack paths.
- MobSF and manual testing give an objective, repeatable security score.
- Before-and-after results prove a measurable drop in vulnerabilities.
- The framework can be reused to audit other mobile apps.

REFERENCES

- 1. G. S. Tuncay, “Android Permissions: Evolution, Attacks, and Best Practices,” IEEE Security & Privacy, 2024.
- 2. R. Li, W. Diao, and D. Gao, “Custom Permission Misconfigurations in Android: A Large-Scale Security Analysis,” in Proc. IEEE TrustCom, 2024.
- 3. A. Barzolevskaia, E. Branca, and N. Stakhanova, “Measuring and Characterizing (Mis)compliance of the Android Permission System,” IEEE Transactions on Software Engineering, 2024.
- 4. X. Hu et al., “TySA: Enforcing Security Policies for Safeguarding Against Permission-Induced Attacks in Android Applications,” IEEE Access, vol. 12, 2024.
- . T. Fedynyshyn, O. Mykhaylova, and I. Opirskyy, “Security Implications of Mobile Development Frameworks,” IEEE TCSET, 2024.



SIMATS ENGINEERING



SIMATS

Saveetha Institute of Medical And Technical Sciences

(Declared as Deemed to be University under Section 3 of UGC Act 1956)

Thank You

Secure Mobile Application Framework

Hari Sai | Dinesh